



ECE593 – Fundamentals of Pre-Silicon Validation

Verification of an AHB2APB Bridge

Milestone 5

Team Members:

Daniel Jacobsen(dj23@pdx.edu): 905696959

Balaji Ginkal Harisha(balajig@pdx.edu): 952724638

Mohith Kumar Bennahatti Chikkegowda(mohithb@pdx.edu): 902084859

Akshaya Kudumalakunte Ravi Kumar(akshayak@pdx.edu): 969375449

Introduction

The project of design and verification of AHB2APB is done using System Verilog and UVM based testbench development methodologies to create Verification IP components of the design.

The APB2APB bridge is a crucial component in SOC architectures as it enables communication between the high-performance AHB(Advanced High-Performance Bus) and the low-performance APB(Advanced Peripheral Bus).

The RTL code for our project is chosen from the github: <https://github.com/prajwalgekkouga/AHB-to-APB-Bridge>

The tool used to compile the design files and to verify is QuestaSim. QuestaSim was chosen as it provides full support for System Verilog and UVM.

AHB2APB Bridge

The AHB2APB bridge is a crucial interface that enables seamless communication between the Advanced High-Performance Bus (AHB) and the Advanced Peripheral Bus (APB) in AMBA-based system-on-chip (SoC) designs. The AHB is a high-speed, pipelined bus used for performance-critical components, while the APB is a simpler, low-power bus designed for peripherals like timers and UARTs. Since these buses have different architectures and operating mechanisms, the AHB2APB bridge ensures proper data transfer by converting AHB transactions into APB transactions while handling synchronization, protocol adaptation, and control signals efficiently.

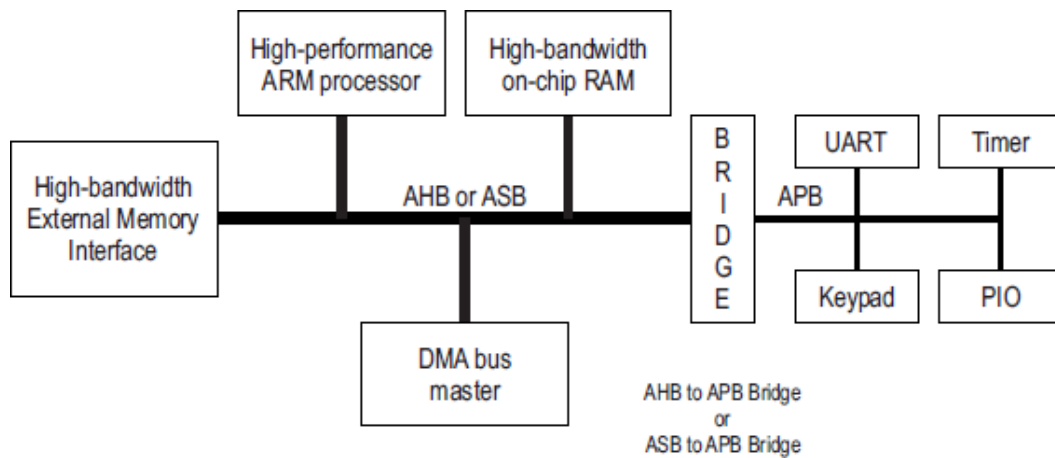


Fig: AHB2APB Bridge

AHB Bus Protocol

AHB is a new generation of AMBA bus which is intended to address the requirements of high-performance synthesizable designs. It is a new level of bus that sits above APB and implements the features required for high performance and high-clock frequency systems as: Burst functions, Split transactions, Single-cycle bus MASTER handover, Single clock edge operation, Non-tristate implementation, Wider data, configurations(64/128 bits), Larger overhead of approximately 27 control signals, upto 75 MASTERS are allowed, Split transaction phases: Address, data(Pipelined), HREADY signal allows insertion of wait-states

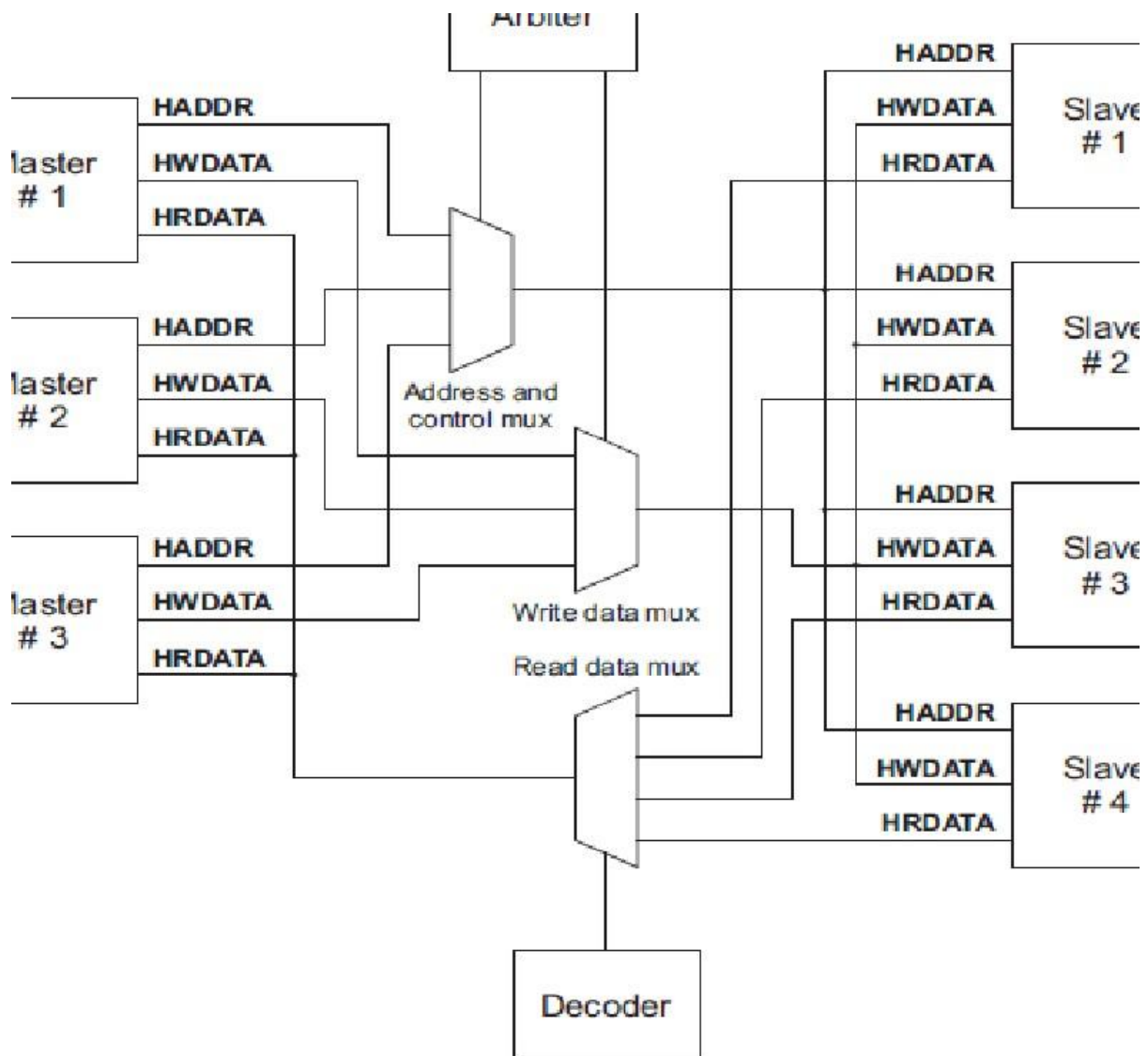


Fig: AHB Architecture

In the above figure, it is depicted that if any MASTER wants to send data, it has to first send request to Arbiter, that gives grant to any 1 of MASTERS and provides select signals to address and write data for MUX. According to this data from that particular MASTER will be reaching the SLAVE.

Decoder will generate the SLAVE Select and decodes the address sent by MASTER and generates the select signal for the SLAVE which is given to read data mux.

The following table gives the description of the signals used in AHB:

Signal Name	Description
HCLK(Bus clock)	This clock times all the bus transfers. All signal timings are related to the posedge.
HRESETn	Active LOW and used to reset the system and the bus.
HADDR[31:0]	Indicates the type of current transfer which may be Non-sequential, Sequential, Idle, Busy
HWRITE[1:0]	If 1, indicates as write transfer else read transfer
HSIZE[2:0]	Indicates size of write transfer i.e., 8 or 16 or 32-bit
HBURST[2:0]	Indicates if transfer forms part of a burst. 4, 8 or 16 beat bursts are supported and burst may be incrementing or wrapping.
HWDATA[31:0]	To transfer data from the MASTER to the bus SLAVES during write operations
HRDATA[31:0]	To transfer data from bus SLAVES to bus MASTER during read operations
HREADY	If 1, it indicates transfer has finished on the bus and if 0, indicates as to extend a transfer
HRESP	Transfer response provides additional information on the status of a transfer, which has 4 different responses like OKAY, ERROR, RETRY, SPLIT
HTRANS[1:0]	It indicates the type of transfer ie., IDLE, BUSY, NON-SEQ, SEQUENTIAL

APB Bus Protocol

APB bus is a overhead which has only 4 control signals. Only 1 MASTER is allowed. It consists of 3 states such as: IDLE, SETUP, ENABLE SLAVE is non-responsive, the transfer takes 2 cycles Makes timing easy to design, as data is latched between 2 cycles.

The following table gives the description of the signals used in APB:

Signals	Description
PCLK	Times all the bus transfers. Both the low and high phase of PCLK is used to control signals
PRESETn	Active LOW Asynchronous reset
PENABLE	It is used to time all accesses on the peripheral bus. It indicates 2 nd cycle of the APB transfer.
PWRITE	If it is 1, then it indicates APB write else if 0 it is APB read.
PRDATA[31:0]	It is driven by a peripheral bus bridge unit during write cycles.
PSELx	A signal from the secondary decoder, within the peripheral bus bridge unit, to each peripheral bus SLAVEx.
PADDR[31:0]	Address bus from MASTER to SLAVE, can be up to 32 bits
PWDATA[31:0]	Write data from MASTER to SLAVE, can be upto 32- bits
PREADY	It is used by the SLAVE to include wait states in the transfer i.e., whenever SLAVE is not ready to complete the transaction, it will request the MASTER for some time by de-asserting the PREADY

AHB2APB Bridge

AHB2APB bridge is an AHB SLAVE. It acts like an interface between high-speed AHB and low-power APB. The read and write transfers on AHB are converted into equivalent APB transfers and AHB is a pipelined protocol and APB is not pipelined. Therefore, for synchronization, this bridge adds WAIT states during the transfers to and from APB and AHB waits for APB.

AHB bus is MASTER, which drives all the signals except HRDATA and HREADY. AHB2APB bridge drives all other signals on APB bus. The APB bus is a SLAVE, which drives PRDATA.

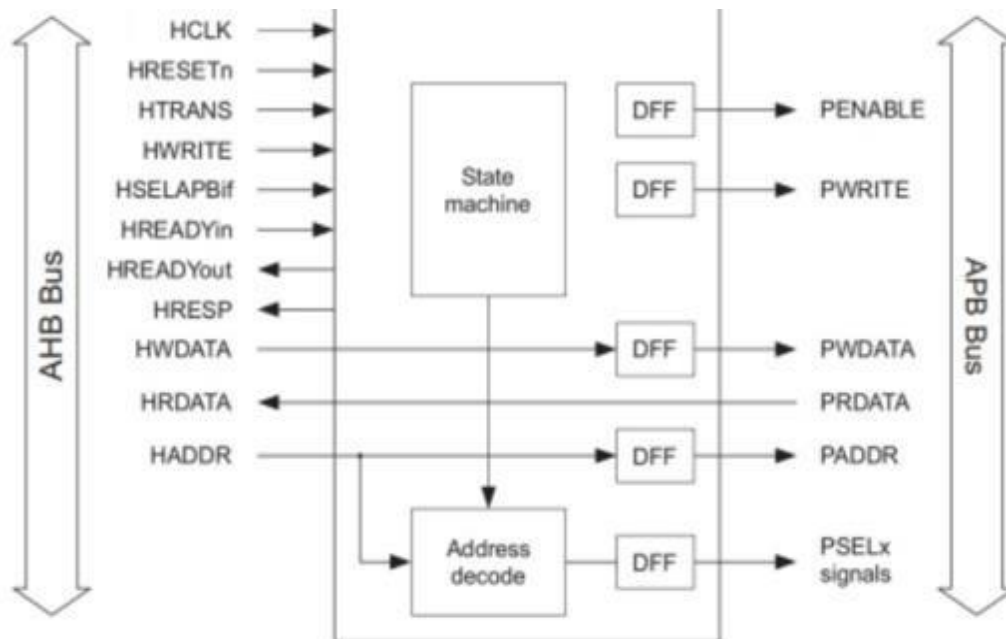


Fig. AHB2APB Block Diagram

The bridge uses Address Decoder for generating PSEL signal.

The flipflops are used because whenever SLAVE inserts WAIT state. MASTER will wait until APB is ready. At that time same address and control information should be held by the bridge. Hence, D-ffs are used.

Some of the bridge functions are as follows:

- Latches the address and holds it valid throughout the transfer.
- It decodes the address and generates a peripheral select-PSEL.
- It drives the data onto APB for write transfer.
- For read transfer, it drives APB data onto the system bus.
- Generates a timing strobe i.e., PENABLE for transfer

During write transfer, the bridge converts HRDATA into PWDATA and during read transfer, PRDATA is converted into HRDATA.

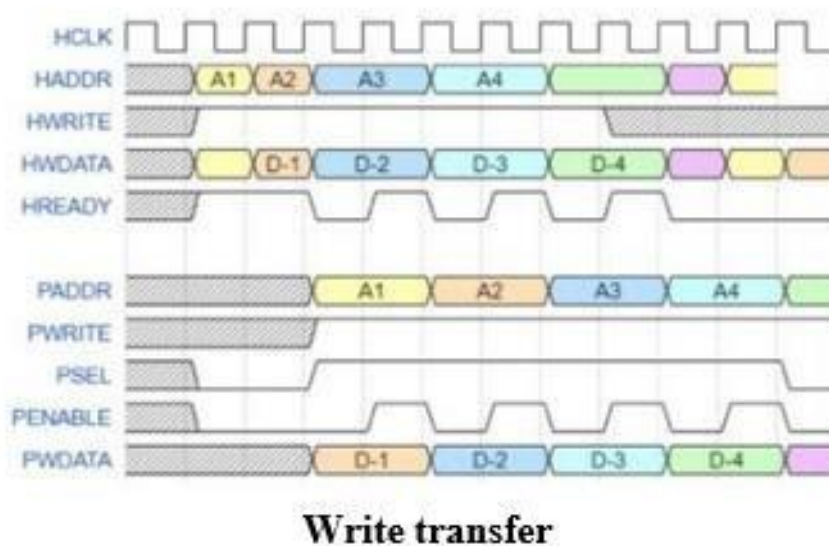
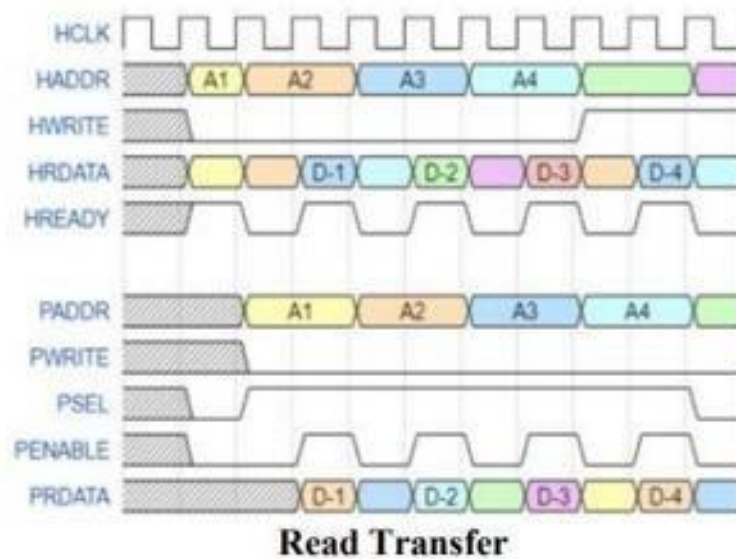


Fig: Read and Write Transfer Waveform

Write Transfer:

The AHB MASTER send 1st address and control information in the 1st cycle. The bridge samples immediately in next cycle and will be latched into local registers and in next cycle, MASTER send address and data for the previous address and again bridge samples in the immediate next posedge..

The currently sampled address is latched into local registers on APB bus i.e., PADDR and PWDATA.

Once PWRITE and PWDATA are on APB bus, it needs 2 cycles to complete the transfer. In the 2nd cycle only, the bridge will generate PENABLE and the transfer on APB is completed, after only bridge can sample new address and the previous data. Therefore, bridge can insert WAIT state by making HREADY = 0 and wait for APB peripheral to complete the transfer.

Read Transfer:

First MASTER send address which is sampled by bridge in next cycle. Once sampled, the same address and write is reflected as PADDR and PWRITE. Once, there are available on APB bus, it requires 2 cycles to complete the transfer. Therefore, in 2nd cycle, bridge generates the strobe signal PENABLE, where PRDATA is available on APB side until 1st transfer is completed. Here, bridge cannot sample next address, so insert WAIT state by making HREADY = 0. So that it can wait for previous transfer to complete. Therefore, the same address at that cycle will get extended and in next cycle, HREADY = 1, because in cycle transfer on APB side is completed.

Therefore, bridge can sample address in the next cycle and HREADY = 1.

Once address is sampled, that will be reflected as PADDR in APB bus and again needs 2 cycles to complete transfer. Therefore, bridge cannot sample the address immediately in the next cycle and insert WAIT state and wait for APB to complete transfer. Whenever PRDATA is reflected onto HRDATA, it won't take any time.

UVM-Based Verification IP Development:

In UVM, the verification environment is structured around reusable components that follow a standardized methodology. The verification of the AHB2APB bridge using UVM involves the following components:

1. UVM Testbench Hierarchy

The UVM testbench is structured hierarchically, with the following key components:

- **UVM Test:** The top-level test class that configures the environment and runs the simulation.
- **UVM Environment:** Contains all the verification components (agents, scoreboard, coverage collector, etc.).
- **UVM Agents:** Each agent represents a bus interface (AHB or APB) and contains a driver, monitor, and sequencer.
- **UVM Scoreboard:** Compares expected and actual transactions.
- **UVM Sequences:** Define the stimulus generation for the AHB and APB interfaces

2. UVM Agents

UVM agents are used to model the behavior of the AHB and APB buses. Each agent consists of the following sub-components:

- **AHB Agent:**
 - **AHB Driver:** Drives AHB transactions (read/write) to the AHB2APB bridge.
 - **AHB Monitor:** Captures AHB transactions and sends them to the scoreboard for checking.
 - **AHB Sequencer:** Generates AHB sequences (e.g., read/write bursts, idle cycles) and sends them to the driver.
- **APB Agent:**
 - **APB Driver:** Responds to APB transactions from the AHB2APB bridge.
 - **APB Monitor:** Captures APB transactions and sends them to the scoreboard for checking.
 - **APB Sequencer:** Generates APB sequences (e.g., read/write responses) and sends them to the driver.

3. UVM Scoreboard

The scoreboard is a critical component in UVM that compares the expected transactions (generated from the AHB side) with the actual transactions (observed on the APB side). It ensures that the AHB2APB bridge correctly translates AHB transactions into APB transactions.

- The scoreboard receives transactions from the AHB monitor and APB monitor.
- It uses a reference model or prediction logic to generate expected APB transactions based on the AHB transactions.
- It compares the expected and observed transactions and reports mismatches.

4. UVM Sequences

UVM sequences are used to generate stimulus for the AHB and APB interfaces. There are two types of sequences:

- **Directed Sequences:**
 - Focus on specific functionalities, such as:
 - Basic read/write operations.
 - Reset and initialization sequences.
 - Corner cases (e.g., back-to-back transactions, wait states).
- **Random Sequences:** Generate constrained random stimulus to explore a wide range of scenarios, including:
 - Random AHB burst types and sizes.

UVM Verification Flow

1. **Build Phase:** Construct the UVM environment, including agents, scoreboard, and coverage collector.
2. **Connect Phase:** Connect the components (e.g., monitors to scoreboard, drivers to sequencers).
3. **Run Phase:** Execute sequences (directed or random) to stimulate the AHB2APB bridge.
4. **Check Phase:** The scoreboard compares expected and observed transactions.
5. **Report Phase:** Generate coverage and simulation reports.

Test Cases for AHB2APB Bridge Verification

The following table outlines the test cases designed to verify the functionality of the AHB2APB bridge. These tests ensure that the bridge correctly handles both **single** and **burst** transactions for **read** and **write** operations.

Test Class	Transaction Type	Description
ahb_apb_burst_write_test	Burst Write	Tests burst write operations on the AHB and APB buses.
ahb_apb_burst_read_test	Burst Read	Tests burst read operations on the AHB and APB buses.
ahb_apb_single_write_test	Single Write	Tests single write operations on the AHB and APB buses.
ahb_apb_single_read_test	Single Read	Tests single read operations on the AHB and APB buses.
ahb_apb_all_test	All Transactions	Tests all transaction types (single/burst, read/write) in a single simulation.

Simulation Results

1. Single Read/Write: Basic read/write operations.

- **Write operation:**

```
# === Scoreboard Summary ===
# AHB Packets: 9
# APB Packets: 9
# Verified Transactions:      4
# Unverified Transactions:    5
# === Coverage Report ===
# RESET: 100.000000%
# WRITE: 100.000000%
# READ: 100.000000%
# === End of Summary ===
#
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 60
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [AHB_SEQUENCE_ITEM] 3
# [AHB_SINGLE_WRITE] 5
# [APB_SINGLE_WRITE] 5
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [UVMTOP] 1
# [ahb_apb_scoreboard] 18
# [ahb_driver] 3
# [ahb_monitor] 9
# [apb_driver] 3
# [apb_monitor] 9
```

- **Read operation:**

```
# === Scoreboard Summary ===
# AHB Packets: 11
# APB Packets: 11
# Verified Transactions:      8
# Unverified Transactions:    3
# === Coverage Report ===
# RESET: 100.000000%
# WRITE: 0.000000%
# READ: 100.000000%
# === End of Summary ===
#
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 71
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [AHB_SEQUENCE_ITEM] 4
# [AHB_SINGLE_READ] 6
# [APB_SINGLE_READ] 5
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [UVMTOP] 1
# [ahb_apb_scoreboard] 22
# [ahb_driver] 4
# [ahb_monitor] 11
# [apb_driver] 3
# [apb_monitor] 11
```

2. Burst Read/Write: Burst mode transactions.

```
# === Scoreboard Summary ===
# AHB Packets: 510
# APB Packets: 510
# Verified Transactions:      257
# Unverified Transactions:    253
# === Coverage Report ===
# RESET: 100.000000%
# WRITE: 0.000000%
# READ: 100.000000%
# === End of Summary ===
#
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 4562
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [AHB_BURST_READ] 505
# [AHB_SEQUENCE_ITEM] 503
# [APB_BURST_READ] 504
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [UVMTOP] 1
# [ahb_apb_scoreboard] 1020
# [ahb_driver] 503
# [ahb_monitor] 510
# [apb_driver] 502
# [apb_monitor] 510
```

```
# === Scoreboard Summary ===
# AHB Packets: 509
# APB Packets: 509
# Verified Transactions:      504
# Unverified Transactions:    5
# === Coverage Report ===
# RESET: 100.000000%
# WRITE: 100.000000%
# READ: 100.000000%
# === End of Summary ===
#
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 4560
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [AHB_BURST_WRITE] 505
# [AHB_SEQUENCE_ITEM] 503
# [APB_BURST_WRITE] 505
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [UVMTOP] 1
# [ahb_apb_scoreboard] 1018
# [ahb_driver] 503
# [ahb_monitor] 509
# [apb_driver] 503
# [apb_monitor] 509
```

3. Combined Cases: Combined read/write and burst read/write.

```
# UVM_INFO apb_sequence.sv(142) @ 20275: uvm_test_top.env_h.apb_agent_h.sequencer_h@@apb_burst_rd [APB_BURST_READ] Completed apb_burst_read_sequence
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 20275: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
# === Scoreboard Summary ===
# AHB Packets: 2027
# APB Packets: 2027
# Verified Transactions:      2019
# Unverified Transactions:      8
# === Coverage Report ===
# RESET: 100.000000%
# WRITE: 100.000000%
# READ: 100.000000%
# === End of Summary ===
#
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :13190
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [AHB_BURST_READ] 505
# [AHB_BURST_WRITE] 505
# [AHB_SEQUENCE_ITEM] 1013
# [AHB_SINGLE_READ] 6
# [AHB_SINGLE_WRITE] 5
# [APB_BURST_READ] 504
# [APB_BURST_WRITE] 505
# [APB_SINGLE_READ] 5
# [APB_SINGLE_WRITE] 5
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [UVMTOP] 1
# [ahb_apb_scoreboard] 4054
# [ahb_driver] 1013
# [ahb_monitor] 2027
# [apb_driver] 1011
# [apb_monitor] 2027
# ** Note: $finish : /pkgs/mentor/questa/2021.3_1/questasim/linux_x86_64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 20275 ns Iteration: 61 Instance: /tb_top
# End time: 21:21:48 on Mar 04,2025, Elapsed time: 0:00:16
# Errors: 0, Warnings: 0
```